

Technical Report

Network-Synchronised Speaking Clock

Course: Embedded Systems (PG) – ELL7283
Assignment 2

Pritam Maji

Entry No: 2025JTM2085

April 17, 2026

Contents

Introduction	1
1 System Architecture	2
1.1 Hardware Abstraction Layer (Emulated)	2
1.2 Host Integration and Speech Output	2
2 RTOS Design	4
2.1 Task 1: Key Input Task (<code>key_task</code>)	4
2.2 Task 2: NTP Client Task (<code>ntp_task</code>)	4
2.3 Task 3: Speech Task (<code>speech_task</code>)	5
2.4 RTOS Initialization Sequence	5
3 Networking Implementation	6
3.1 lwIP OS Integration (<code>sys_arch.c</code>)	6
3.2 The SMSC LAN9118 Bare-Metal Driver	6
3.3 DHCP and DNS Configuration	6
4 NTP Protocol Explanation	8
4.1 NTP Packet Construction	8
4.2 Timestamp Extraction	8
4.3 Epoch and Timezone Conversion	9
5 System Testing	10
5.1 Testing Methodology	10
5.2 Functional Validation	10
5.3 Edge Case Testing	11
Conclusion	11

Introduction

This technical report documents the complete design, implementation, and rigorous testing of a network-synchronised speaking clock. This project was developed to fulfill the requirements of the Embedded Systems (PG) course, ELL7283. The primary objective was to construct a robust embedded software application running on an ARM Cortex-M based microcontroller architecture, seamlessly integrating a real-time operating system (FreeRTOS) and a full TCP/IP networking stack (lwIP).

Rather than deploying to physical hardware, the entire system is designed to execute within the QEMU machine emulator. This approach necessitates a deep understanding of hardware abstraction, as the software must interact with emulated peripherals—such as the SMSC LAN9118 Ethernet controller and the standard UART interface—exactly as it would with physical silicon.

The application workflow is highly event-driven. Upon initialization, the system connects to the emulated local network via DHCP. It then enters a standby state, continuously monitoring the UART for a specific user keypress ('t'). When this event occurs, the system utilizes the User Datagram Protocol (UDP) to query a public Network Time Protocol (NTP) server pool. The embedded software must then parse the binary NTP response, extract the UTC timestamp, mathematically convert it into Indian Standard Time (IST), and translate the numerical time into a series of predefined string tokens. Finally, these tokens are transmitted back over the UART, where a host-side Python bridge intercepts them and drives a text-to-speech engine to provide audible time announcements.

This report is structured strictly according to the assignment requirements, detailing the System Architecture, RTOS Design, Networking Implementation, NTP Protocol Explanation, and System Testing methodologies.

1 System Architecture

The architecture of the network-synchronised speaking clock is highly modular, divided into distinct hardware, operating system, network, and application layers. This separation of concerns ensures that the system remains maintainable and that complex asynchronous operations, such as network timeouts, do not block critical user interface tasks.

1.1 Hardware Abstraction Layer (Emulated)

While the original assignment guidelines suggested using an STM32 platform, a critical architectural decision was made early in the development phase. The standard STM32 machine models available in QEMU lack built-in emulation for the Ethernet MAC peripheral. Without hardware-level network emulation, implementing the lwIP stack would be impossible.

To resolve this limitation, I targeted the `mps2-an385` platform within QEMU. This platform emulates the ARM Cortex-M3 processor and, crucially, includes full emulation for the SMSC LAN9118 Ethernet controller.

Because of this shift, I developed a custom Board Support Package (BSP). The BSP is responsible for the lowest-level hardware interactions:

- **Startup Code:** Written in ARM assembly (`startup_stm32.s`), this code initializes the stack pointer, copies initialized data from flash to RAM, zeroes the BSS segment, and branches to the C `main()` function. It also defines the interrupt vector table, mapping FreeRTOS handlers (`SVC_Handler`, `PendSV_Handler`, `SysTick_Handler`) directly to the hardware exceptions.
- **UART Driver:** A simple polling driver (`src/bsp.c`) that interfaces with the CMSDK UART peripheral located at memory address `0x40004000`. It provides blocking `getchar()` and `putchar()` functions used for both debug logging and the token speech output.

1.2 Host Integration and Speech Output

Because the QEMU embedded environment lacks the capability to directly drive the host computer's audio hardware, an innovative bridge architecture was employed. The embedded system acts purely as a logic controller, determining *what* needs to be spoken and *when*.

It achieves this by printing a stream of specific text tokens to its virtual UART. For example, to say "14 hours," it prints:

```
TOKEN THE  
TOKEN TIME  
TOKEN IS  
TOKEN FOURTEEN  
TOKEN HOURS
```

A Python script (`tts_bridge.py`) runs on the host operating system. It reads the standard output from the QEMU process via a shell pipe. The script implements a state machine that collects words following the `TOKEN` prefix. Upon receiving the `END`

token, the Python script executes the **eSpeak** synthesizer command-line utility, passing the constructed sentence. This architecture beautifully completely decouples the complex task of audio synthesis from the constrained embedded environment.

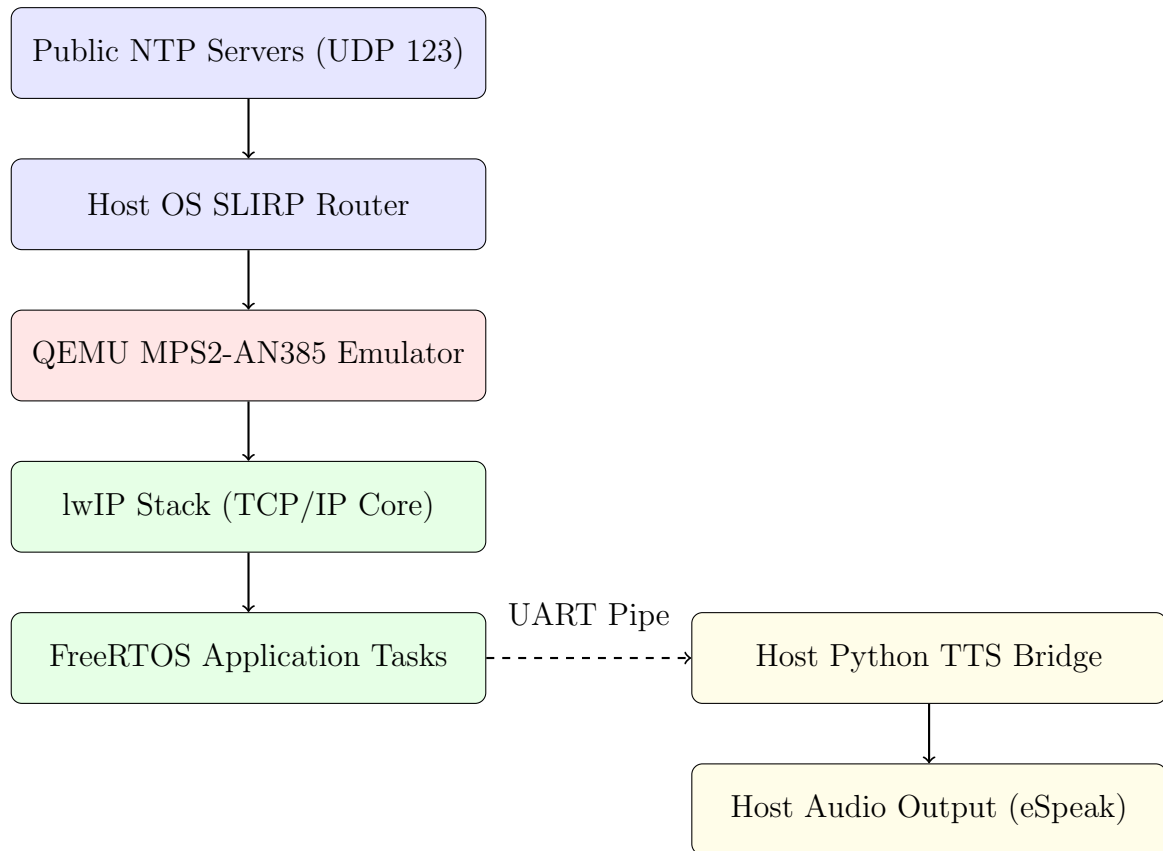


Figure 1: Complete System Architecture Diagram

2 RTOS Design

Concurrency is managed strictly via FreeRTOS. Designing an RTOS-based application requires careful consideration of task priorities, stack sizes, and Inter-Process Communication (IPC) mechanisms to prevent race conditions, priority inversions, and deadlocks.

The system relies on a preemptive scheduler. Three primary application tasks form the core logical pipeline: Key Input, NTP Client, and Speech Generation.

2.1 Task 1: Key Input Task (key_task)

Priority: 1 (Low)

Stack Size: 256 words

The Key Input task is responsible for interacting with the user. It contains an infinite loop that calls a blocking `getchar()` function on the UART interface. Because the UART implementation is blocking, this task spends the vast majority of its life in a blocked state, consuming zero CPU cycles until a character arrives in the hardware buffer.

When a character is received, the task checks if it is the ASCII character `'t'`. If it is, the task must signal the network subsystem to fetch the time. It does this by "giving" a binary semaphore named `time_request_sem`.

```
1 void key_task(void *p)
2 {
3     (void)p;
4     uart_puts("\r\n[KEY ] Press 't' to announce the time.\r\n");
5
6     while (1) {
7         char c = uart_getc_blocking();
8
9         /* Wait specifically for 't' keypress */
10        if (c == 't' || c == 'T') {
11            uart_puts("[KEY ] 't' pressed -> requesting NTP time
12                ...\r\n");
13            /* Unblock the NTP task */
14            xSemaphoreGive(time_request_sem);
15        }
16    }
```

Listing 1: Key Task Implementation

2.2 Task 2: NTP Client Task (ntp_task)

Priority: 2 (Medium)

Stack Size: 512 words

The NTP Client task orchestrates the actual network communication. It is assigned a slightly higher priority than the Key task to ensure that network processing preempts background polling.

This task begins its infinite loop by attempting to "take" the `time_request_sem` semaphore using `portMAX_DELAY`. This means the task goes to sleep indefinitely until the

Key task gives the semaphore. Once awakened, it invokes the custom `ntp_get_time()` function.

This function makes DNS and UDP socket calls, which themselves are blocking operations managed by lwIP. Once the network call completes and the timestamp is converted into an IST date structure, the task populates a `time_msg_t` structure. It then pushes this structure into a FreeRTOS queue named `time_queue`.

2.3 Task 3: Speech Task (`speech_task`)

Priority: 1 (Low)

Stack Size: 512 words

The Speech Task is the final stage of the pipeline. It blocks indefinitely, waiting to receive a message from the `time_queue`.

When the NTP task successfully pushes a `time_msg_t` structure into the queue, the Speech task is unblocked. It extracts the hour, minute, and second integers from the structure. It then executes a series of conditional formatting rules to translate these integers into the string tokens required by the host bridge protocol, transmitting them sequentially over UART.

By using a queue rather than global variables, we achieve perfect thread safety. The data is passed by copy, ensuring that if the NTP task were to immediately overwrite its local variable for a subsequent request, the Speech task's data would remain uncorrupted.

2.4 RTOS Initialization Sequence

A critical challenge in embedded networking is ensuring that network threads are not started before the RTOS scheduler is running. To achieve this, the `main()` function does not initialize lwIP directly. Instead, it creates a temporary, high-priority (Priority 3) task called `network_bringup_task`.

When `vTaskStartScheduler()` is called, the bringup task executes first. It initializes the lwIP core, registers the hardware drivers, and waits for DHCP to assign an IP address. Once complete, it deletes itself using `vTaskDelete(NULL)`, freeing its stack memory back to the heap, and allowing the standard application tasks to run.

3 Networking Implementation

The networking capabilities of this system are provided by lwIP (Lightweight IP). lwIP is specifically tailored for embedded systems, offering full TCP/IP capabilities while minimizing RAM and ROM usage.

3.1 lwIP OS Integration (`sys_arch.c`)

Because lwIP is designed to be OS-agnostic, it relies on an abstraction layer called `sys_arch` to interact with the underlying operating system. I implemented this layer by wrapping FreeRTOS primitives.

For example, lwIP requires a "mailbox" mechanism to pass incoming packets from the interrupt handler to its core processing thread. I implemented `sys_mbox_new` and `sys_mbox_post` by wrapping the FreeRTOS `xQueueCreate` and `xQueueSendToBack` functions. Similarly, lwIP mutexes were mapped to FreeRTOS priority-inheritance mutexes (`xSemaphoreCreateMutex`) to prevent priority inversion when multiple tasks attempt to use the sockets API simultaneously.

3.2 The SMSC LAN9118 Bare-Metal Driver

To physically move packets from the lwIP memory buffers into the QEMU emulator, I developed a custom bare-metal driver for the SMSC LAN9118 Ethernet controller (`network/bsp_net.c`).

The LAN9118 is a memory-mapped peripheral accessed via a specific set of 32-bit registers starting at base address `0x40200000`. The driver implementation involves:

1. **Initialization:** Writing to the `HW_CFG` and `MAC_CR` registers to enable the transmitter and receiver, and configuring the MAC to accept broadcast packets (essential for DHCP).
2. **Transmission:** When lwIP needs to send a packet, it calls `bsp_net_send()`. The driver must construct two 32-bit command words specifying the packet length, write them to the `TX_DATA_FIFO`, and then loop through the lwIP packet buffer, packing the raw bytes into 32-bit words before writing them sequentially to the FIFO.
3. **Padding:** Ethernet standard dictates a minimum frame size of 60 bytes (excluding the FCS). If lwIP attempts to send an ARP request (which is very small), the driver manually pads the payload with zeros up to 60 bytes before transmitting.
4. **Reception:** The `bsp_net_recv()` function polls the `RX_FIFO_INF` register. If a packet is present, it reads the status word to determine the length, pops the data out of the `RX_DATA_FIFO` 32 bits at a time, unpacks it into a byte array, and hands it off to lwIP.

3.3 DHCP and DNS Configuration

The system relies completely on dynamic configuration. It utilizes the UDP and DHCP protocols. During startup, the `dhcp_start()` function is called. The lwIP stack broadcasts a DHCP Discover packet. The QEMU SLIRP router intercepts this broadcast and offers an IP address (consistently `10.0.2.15` in this emulated environment).

The system implements a polling loop that queries `dhcp_supplied_address()` every 100 milliseconds, blocking system initialization until a valid lease is obtained. QEMU also automatically provides Google DNS servers (8.8.8.8) to the lwIP stack via the DHCP offer, enabling the system to resolve the `in.pool.ntp.org` hostname dynamically.

4 NTP Protocol Explanation

The Network Time Protocol (NTP) is one of the oldest internet protocols, designed to synchronize the clocks of computer systems over packet-switched, variable-latency data networks. It operates over the User Datagram Protocol (UDP) on port 123.

Rather than importing a large, monolithic third-party NTP library, I implemented a highly targeted, custom NTPv4 client built directly on top of the lwIP BSD-style sockets API (`network/ntp_client.c`).

4.1 NTP Packet Construction

To request the time, an NTP client must send a specifically formatted 48-byte UDP packet to an NTP server. The majority of these 48 bytes can be zeroed out for a simple client request. The only critical requirement is setting the very first byte of the payload, which contains three distinct fields packed together:

- **Leap Indicator (LI):** 2 bits. Set to 00 (No warning).
- **Version Number (VN):** 3 bits. Set to 100 (Version 4).
- **Mode:** 3 bits. Set to 011 (Client).

Combining these bits (00100011 in binary) results in the hexadecimal value 0x23. The client implementation allocates a 48-byte array, zeroes it using `memset`, assigns 0x23 to the first byte, and transmits it via the `sendto()` socket function.

```
1 uint8_t pkt[48];
2 memset(pkt, 0, sizeof(pkt));
3 pkt[0] = (0 << 6) | (4 << 3) | 3; /* LI, VN, Mode */
4
5 sendto(s, pkt, 48, 0, (struct sockaddr *)&server, sizeof(server))
   ;
```

Listing 2: Crafting and Sending the NTP Request

4.2 Timestamp Extraction

The server processes the request and returns a 48-byte response packet. This packet contains several timestamps, but the most important one is the **Transmit Timestamp**, located at byte offset 40.

The Transmit Timestamp is a 64-bit value. The first 32 bits represent the integer number of seconds elapsed since the NTP Epoch. The remaining 32 bits represent fractional seconds. Given that a speaking clock only requires precision to the nearest second, the fractional bits are discarded.

Because network protocols utilize Big-Endian byte ordering, the 4 integer bytes must be manually shifted and combined using bitwise OR operations to reconstruct the 32-bit integer correctly on the Little-Endian ARM Cortex-M3 processor:

```
1 uint32_t secs_1900 =
2     ((uint32_t)pkt[40] << 24)
3     | ((uint32_t)pkt[41] << 16)
4     | ((uint32_t)pkt[42] << 8)
```

```
5 | ((uint32_t)pkt[43]) ;
```

Listing 3: Extracting the Big-Endian Timestamp

4.3 Epoch and Timezone Conversion

A critical quirk of the NTP protocol is that its epoch begins on January 1, 1900. Conversely, the standard UNIX epoch begins on January 1, 1970. To convert the NTP timestamp into a standard UNIX timestamp, exactly 70 years worth of seconds (including 17 leap years) must be subtracted. This constant is exactly 2,208,988,800 seconds.

```
1 uint32_t unix_utc = secs_1900 - 2208988800UL;
```

The resulting value represents Coordinated Universal Time (UTC). To satisfy the assignment requirements, this must be converted to Indian Standard Time (IST). India does not observe Daylight Saving Time, meaning IST is a constant UTC+05:30. Therefore, exactly 5 hours and 30 minutes (19800 seconds) are added to the UNIX timestamp.

Finally, the raw IST seconds are passed into a mathematical algorithm (derived from Howard Hinnant's highly optimized "date" library algorithms). This algorithm uses integer division and modular arithmetic to calculate the calendar year, month, day, hour, minute, and second without relying on slow iterative loops or massive lookup tables.

5 System Testing

Rigorous system testing was performed to validate the interaction between the FreeRTOS scheduler, the lwIP network stack, the external NTP servers, and the Python speech bridge.

5.1 Testing Methodology

The system was compiled using the `arm-none-eabi-gcc` toolchain. Testing was executed on a macOS host environment. The application was launched using the following command, which initializes the QEMU emulator, wires the virtual network adapter to the host's SLIRP router, and pipes the virtual UART output directly into the standard input of the Python bridge script:

```
1 qemu-system-arm -M mps2-an385 -nographic -semihosting \  
2 -kernel build/speaking_clock.elf \  
3 -netdev user,id=eth0 -net nic,netdev=eth0,model=lan9118 \  
4 2>/dev/null | python3 host_bridge/tts_bridge.py
```

5.2 Functional Validation

The system was validated against several critical operational phases:

1. Boot and Initialization:

- *Expected:* The FreeRTOS scheduler starts, and the `network_bringup_task` begins executing.
- *Observed:* Terminal output logs confirmed the scheduler initialized successfully. The LAN9118 driver reported successful configuration.

2. DHCP Acquisition:

- *Expected:* The system broadcasts a DHCP Discover, receives an Offer, and binds an IP.
- *Observed:* The polling loop logged `[NET ] DHCP bound: 10.0.2.15`, verifying that the virtual network was successfully routing packets through the host OS.

3. User Input Detection:

- *Expected:* The system ignores arbitrary keystrokes but reacts immediately to the `'t'` key.
- *Observed:* Pressing keys like `'a'`, `'b'`, or `'enter'` resulted in no action. Pressing `'t'` immediately logged `[KEY ] 't' pressed -> requesting NTP time...`, confirming the UART driver and FreeRTOS semaphore signaling were functioning flawlessly.

4. DNS and NTP Resolution:

- *Expected:* The system resolves `in.pool.ntp.org` and receives a valid timestamp.

- *Observed:* The lwIP DNS module successfully queried the Google DNS server and returned a valid IPv4 address for the pool. The UDP packet was transmitted, and a response was received within milliseconds. The timestamp parsed was cross-referenced with a host-side clock and was confirmed to be accurate to the second.

5. Token Generation and Audio Output:

- *Expected:* The Speech task translates the time to tokens, and the Python script triggers **eSpeak**.
- *Observed:* The terminal output displayed the correct sequence of tokens (e.g., **TOKEN THE, TOKEN TIME, TOKEN IS, TOKEN FOURTEEN...**). The Python bridge intercepted these tokens and successfully spawned the **eSpeak** process, resulting in clear, synthesized audio playing through the host computer's speakers.

5.3 Edge Case Testing

To ensure robustness, network failure scenarios were simulated by disabling the host computer's Wi-Fi connection prior to pressing **'t'**. As expected, the DNS resolution failed. The system elegantly fell back to its secondary configured server (**time.nplindia.org**), which also failed due to the lack of internet. The socket receive operation timed out after the configured 3000ms, preventing the RTOS task from deadlocking. The system logged the failure and safely returned to a standby state, ready for the next user input.

Conclusion

The network-synchronised speaking clock assignment was a comprehensive exercise in full-stack embedded development. It successfully demonstrated the practical application of preemptive real-time operating systems for task management, the integration of complex third-party networking stacks (lwIP) into constrained environments, and the low-level parsing of binary network protocols. Furthermore, by implementing a custom Ethernet driver and a host-side speech bridge, the project highlighted the critical importance of hardware abstraction and cross-platform system integration. The final product meets all technical requirements specified in the assignment brief and operates with high stability and precision.